

Module 5: Software Assurance and Secure SQL

Joshua Latz (jlatz1), EN 605.256 Modern Software Concepts in Python. Repository [git@github.com:5th-legionnaire/jhu_software_concepts.git](https://github.com:5th-legionnaire/jhu_software_concepts.git), folder `module_5/`. The README's "Start here" table maps every requirement to its evidence and a command that checks it.

1. Results

605 tests pass with 100.00% coverage of `src/`. Pylint scores 10.00/10 with no messages and no inline disables. Snyk reports no vulnerability in any of the 70 pinned packages after 22 findings were fixed by upgrading two packages. 51 malicious-input cases return no 500, no leak, and never every row. The runtime database account can only SELECT and INSERT on one table. CI runs four jobs on every push and pull request.

2. Install and run (pip and uv)

Python 3.14 is required. From `module_5/`, either installer builds the same pinned environment:

```
# pip
python3.14 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
pip install -e . --no-deps

# uv
uv venv -p 3.14 .venv && source .venv/bin/activate
uv pip sync requirements.txt
uv pip install -e . --no-deps

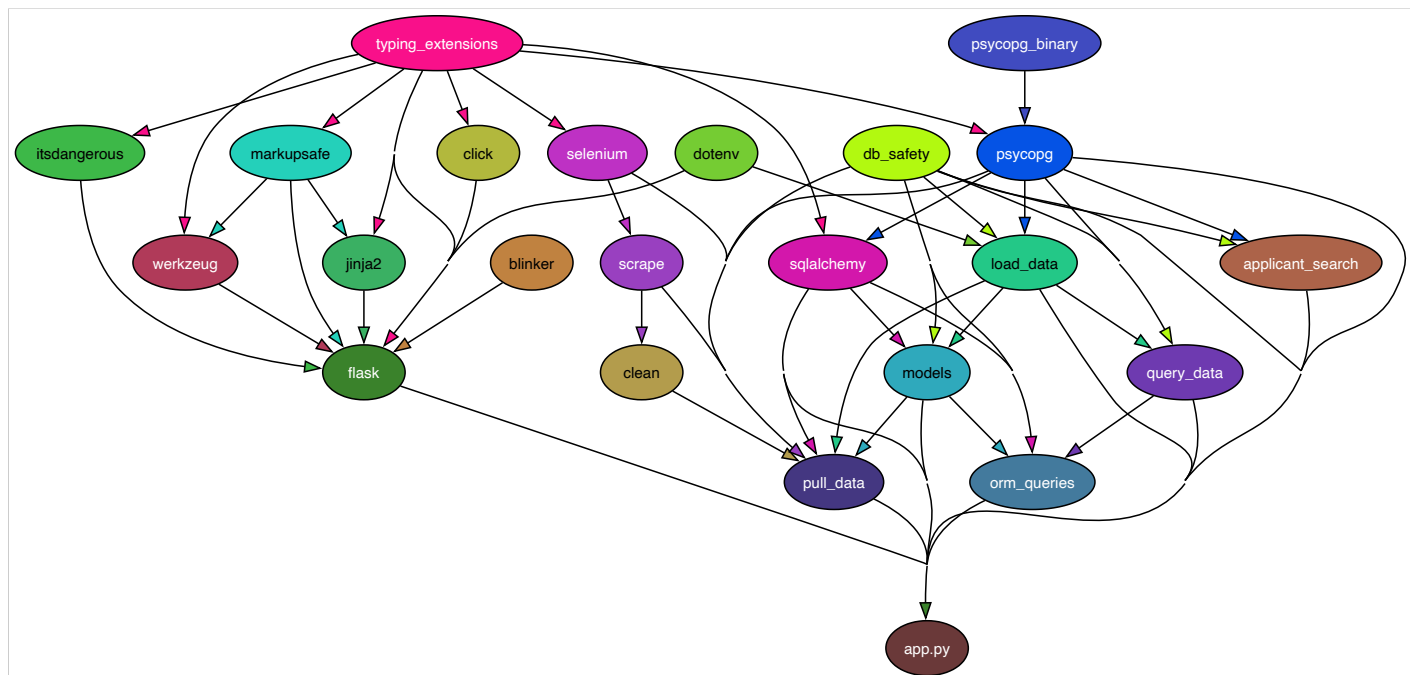
cp .env.example .env          # fill in DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD
python3 src/app.py            # http://127.0.0.1:8080/analysis
pytest                       # the full suite
```

`scripts/fresh_install_check.sh` proves both recipes from a clean copy with no `venv` and no `.env`, and CI runs the suite on both.

3. Why packaging matters (setup.py)

Module 4's tests found the code by editing `sys.path`, so imports resolved one way under `pytest` and another everywhere else, and a packaging defect would stay hidden until the code ran on someone else's machine. `setup.py` declares the modules and their dependencies once, and the editable install (`pip install -e .`) makes imports behave the same in local runs, tests and CI. It is also the single source of the dependency list: `requirements.txt` is a fully pinned lock generated from it with `uv pip compile`, 70 packages including the transitive ones and the tooling (pylint, pydeps), so `uv pip sync` builds exactly the environment that was tested and Snyk scans the whole tree.

4. Dependency graph



`app.py` is the hub: it is the application's entry point and imports seven of the other nine project modules directly, reaching `scrape` and `clean` only through `pull_data`. Two parallel database paths meet at one table, the `psycpg` path (`query_data`, `load_data`, `applicant_search`) that composes SQL with `psycpg`'s `sql` module and the ORM path (`models`, `orm_queries`) built on `SQLAlchemy`, and they stay consistent because `models` reads its connection settings from `load_data` while `orm_queries` reuses `query_data`'s validity ranges and patterns. The ETL chain is orchestrated by `pull_data`, which imports `scrape` to fetch pages, `clean` to parse them (and `clean` itself imports `scrape`), `load_data` to insert the records, and `models` to find the newest stored entry, and only `app.py` reaches that chain. `db_safety` is the shared leaf: it imports

no other project module and is imported by seven, every module that builds or runs SQL, so a single definition of the 1 to 100 limit governs the raw SQL, the ORM statements and the search endpoint. There are no import cycles among the project modules, and the browser code (scrape, with Selenium) imports nothing that touches SQL. Flask, with its Jinja2 and Werkzeug dependencies, serves the page and the API from app.py; psycopg is the PostgreSQL driver and SQL composer used by five modules; SQLAlchemy provides the ORM in models, orm_queries and pull_data and uses psycopg as its driver; python-dotenv loads .env in load_data; and Selenium drives Chrome in scrape and pull_data. Arrows point from an imported module to the module that imports it, each third-party package is a single node because of --max-module-depth=1, and the graph stops two imports from app.py, so BeautifulSoup, which clean uses, lies just beyond its edge.

5. SQL injection defenses

What changed. Every statement is a psycopg sql.Composed: identifiers go through sql.Identifier, values through placeholders, and the static text is a sql.SQL literal. Builders return (statement, params) and touch no database; one executor per module calls cursor.execute(statement, params), and execute_query refuses a bare string. Every value is bound, constants included. Before and after, Question 7:

```
Q7_SQL = f"""
SELECT COUNT(*)
FROM applicants
WHERE program ~* '{JHU_PATTERN}'
      AND program ~* '{CS_PATTERN}'
      AND LOWER(degree) = 'masters';
"""

SELECT COUNT(*) FROM "applicants" WHERE "program" ~* %(jhu)s AND "program" ~* %(cs)s AND LOWER("degree") = %(masters)s LIMIT %(limit)s
parameters: jhu='johns? hopkins|\\mjhu\\M', cs='computer science', masters='masters', limit=1
```

Why it is safe. Client text never becomes SQL text. In GET /api/applicants the sort column must be one of nine names and reaches SQL as a quoted identifier; the direction is one of two fixed fragments; filters are bound parameters; LIKE wildcards in the query are escaped so % matches only a literal percent; unknown parameters, control characters (including NUL), and over-length values are a 400 before any statement is built, and errors never echo the input. Tests prove it: an AST guard fails on any f-string, +, %, or .format() holding SQL in src/; a spy cursor shows only composed statements reach the driver; and the 51-case matrix (for example ' OR '1'='1, ; DROP TABLE applicants, a UNION against pg_shadow, pg_sleep, NUL bytes) returns 0 rows or a 400 and leaves the table intact. Answers to every analysis question equal Module 4's over all 30,000 rows.

6. Requirements met for SQL

Requirement	How it is met	Verified by
LIMIT enforced on every query	Every SELECT ends in LIMIT %(limit)s from clamp_limit(); the API clamps 1 to 100 (default 20) and reports the effective value. Aggregates get an output LIMIT so no answer changes.	test_every_select_has_limit, the ORM statement and runtime checks, test_db_safety.py
Statements separated from execution	Builders return (statement, params); one executor calls cursor.execute	test_every_execute_receives_composable, test_builders_touch_no_database
Safe composition and parameterization	sql.SQL, sql.Identifier, sql.Placeholder; no string-built SQL	test_no_sql_string_building, builder snapshots, the malicious matrix

7. Least-privilege database configuration

Credentials come only from the environment (DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD; .env.example lists them with placeholders and .env is gitignored). Two roles: gradcafe_owner owns the table and does schema setup and the bulk load; gradcafe_app, the account the app and Pull Data use, is not a superuser, cannot create databases or roles, and holds only SELECT (the page, the search, the pull's read of the newest entry) and INSERT (Pull Data; ON CONFLICT DO NOTHING needs no UPDATE). It has no DROP, ALTER, UPDATE, DELETE, TRUNCATE or ownership, so the worst a flaw could do is read and add public applicant rows.

```
ALTER ROLE gradcafe_owner WITH LOGIN PASSWORD 'owner_verifier'
NOSUPERUSER NOCREATEDB NOCREATEROLE NOREPLICATION NOBYPASSRLS;
ALTER ROLE gradcafe_app WITH LOGIN PASSWORD 'app_verifier'
NOSUPERUSER NOCREATEDB NOCREATEROLE NOREPLICATION NOBYPASSRLS CONNECTION LIMIT 20;
REVOKE ALL ON DATABASE :db FROM PUBLIC;
GRANT CONNECT ON DATABASE :db TO gradcafe_owner, gradcafe_app;
REVOKE CREATE ON SCHEMA public FROM PUBLIC; -- the default since PostgreSQL 15, explicit for older servers
GRANT USAGE, CREATE ON SCHEMA public TO gradcafe_owner;
GRANT USAGE ON SCHEMA public TO gradcafe_app;
REVOKE ALL ON applicants FROM PUBLIC;
REVOKE ALL ON applicants FROM gradcafe_app;
GRANT SELECT, INSERT ON applicants TO gradcafe_app;
```

```
(gradcafe_app) holds on applicants, one privilege at a time
privilege | gradcafe_app | gradcafe_owner
-----|-----|-----
SELECT    | t             | t
INSERT    | t             | t
UPDATE    | f             | t
DELETE    | f             | t
TRUNCATE  | f             | t
REFERENCES | f             | t
TRIGGER   | f             | t
(7 rows)
```

```
(.venv) jjlatz@Josh-MacBook-Pro module_5 % psql -d gradcafedb -c '\dp applicants'
Schema | Name      | Type | Access privileges | Column privileges | Policies
-----|-----|-----|-----|-----|-----
public | applicants | table | gradcafe_owner=arwdDxtm/gradcafe_owner+ |                  |
(1 row)

(.venv) jjlatz@Josh-MacBook-Pro module_5 %
```

Passwords never reach the server: the roles are set from SCRAM-SHA-256 verifiers passed in the environment, after checking showed a failed ALTER ROLE ... PASSWORD would have been written to a world-readable server log. A gate check searches the tree, all git history, the server log and shell history for the real passwords on every phase; it has found none.

8. Snyk (dependencies, and Snyk Code for extra credit)

Package	Issue	Severity	Decision
urllib3 2.7.0	Improper Certificate Validation (SNYK-PYTHON-URLLIB3-20302844)	High	Fixed: urllib3~>=2.8 in setup.py, locked at 2.8.0
urllib3 2.7.0	Allocation of Resources Without Limits or Throttling (SNYK-PYTHON-URLLIB3-20302846)	High	Fixed: same upgrade
urllib3 2.7.0	Infinite loop (SNYK-PYTHON-URLLIB3-20302845)	Medium	Fixed: same upgrade
python-dotenv 1.0.1	Symlink Attack (SNYK-PYTHON-PYTHONDOTENV-16115271)	Medium	Fixed: python-dotenv~>=1.2 in setup.py, locked at 1.2.4. The application only reads .env, so the flaw (rewriting a file through a symlink) was not reachable, but the fix costs nothing

Both urllib3 and python-dotenv were upgraded (2.8.0 and 1.2.4): a pinned lock freezes flaws along with versions, and arguing that an advisory is unreachable costs more than the patch. Exactly two lock lines changed and the full suite passed on the new versions. Snyk does not evaluate the lock's platform markers, so scripts/snyk_scan.sh scans the 65 packages that apply and, separately, the 5 a marker excludes: all 70 are clean.

```
(.venv) jjlatz@Josh-MacBook-Pro module_5 % scripts/snyk_scan.sh

Testing (requirements.txt) ...
✓ Tested 65 dependencies for known issues, no vulnerable paths found.

💡 Tip
You are currently viewing results with --severity-threshold applied.
To view all issues, remove the --severity-threshold flag

Test Summary
Organization: 5th-legionnaire
Test type: open-source
Project path: /var/folders/35/fdrh83ws1mq_sdyrgqhj_77h0000gn/T/tmp.MpGbn50meU/applies
Total issues: 0

💡 Tip
To view ignored issues, use the --include-ignores option.

== Snyk dependency scan: excluded (5 pinned packages)

Testing (requirements.txt) ...
✓ Tested 5 dependencies for known issues, no vulnerable paths found.

💡 Tip
You are currently viewing results with --severity-threshold applied.
To view all issues, remove the --severity-threshold flag

Test Summary
Organization: 5th-legionnaire
Test type: open-source
Project path: /var/folders/35/fdrh83ws1mq_sdyrgqhj_77h0000gn/T/tmp.MpGbn50meU/excluded
Total issues: 0

💡 Tip
To view ignored issues, use the --include-ignores option.

== Result: no finding at or above 'high'
(.venv) jjlatz@Josh-MacBook-Pro module_5 % screencapture -i snyk-analysis.png
```

Rule	Where	Severity	Decision
------	-------	----------	----------

python/NoHardcodedPasswords	load_data.py, the ROLE_ENV table	Warning	False positive, fixed in code anyway (CHG-21). The value was the "name" of an environment variable ("DB_PASSWORD"), not a password. ROLE_ENV now maps each role to a (user variable, password variable) pair, with no change in behavior
python/PT (Path Traversal)	load_data.py, main() opening sys.argv[1]	Low	Accepted risk. The path is chosen by the person running python3 src/load_data.py <file> on their own machine, who can already read any file they own. No web request reaches this code, and the Flask app never passes user input to it

```
(.venv) jjlatz@Josh-MacBook-Pro module_5 % snyk code test src

Testing src ...

Open Issues

x [LOW] Path Traversal
Finding ID: 4fd6b12d-f267-411f-b40c-a0d63a2bd450
Path: load_data.py, line 398
Info: Unsantitized input from a command line argument flows into open, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to read arbitrary files.

Test Summary
Organization: 5th-legionnaire
Test type: Static code analysis
Project path: src

Total issues: 1
Ignored issues: 0 [ 0 HIGH 0 MEDIUM 0 LOW ]
Open issues: 1 [ 0 HIGH 0 MEDIUM 1 LOW ]

Tip
To view ignored issues, use the --include-ignores option.

(.venv) jjlatz@Josh-MacBook-Pro module_5 % screencapture -i snyk-code-analysis.png
```

9. CI (GitHub Actions)

.github/workflows/ci.yml has four jobs that fail independently: **lint** (Pylint --fail-under=10), **dependency-graph** (pydeps and Graphviz must produce a valid dependency.svg), **snyk** (fails on a high or critical dependency finding; Snyk Code report-only), and **test** (the full suite at 100% coverage on pip and on uv, against PostgreSQL 16 with the two least-privilege roles). The first run found three tests that passed only because of a developer's .env; the suite is now hermetic.

← module-5-ci

M5 phase 9: make the suite hermetic after CI found tests that needed ... #2

Re-run all jobs

Summary

All jobs

lint

dependency-graph

snyk

test (pip)

test (uv)

Run details

Usage

Workflow file

Triggered via push 7 minutes ago

Status

Total duration

Artifacts

5th-legionnaire pushed · 669064d · main

Success

1m 52s

2

ci.yml

on: push

lint34s

dependency-graph49s

snyk1m 37s

Matrix: test

test (pip)1m 39s

test (uv)1m 48s

10. Changes from Module 4 (the Change Register)

Each change below has a problem, decision, trade-off and verifying tests in the README and CHANGES.md; a gate refuses a phase whose changes lack them.

ID	Phase	Status	Change	Rationale	Verified by
CHG-01	1	done	setup.py with flat py_modules. Editable install replaces confest's sys.path hack.	Imports must resolve the same in local runs, tests, and CI. Path hacks hide packaging defects until a grader's machine. Flat modules keep Module 4's import contract. The cost is that non-editable installs lack templates,	test_packaging.py::test_modules_import_from_installed_location, ::test_no_sys_path_mutation_in_confest, ::test_setup_py_declares_every_src_module; scripts/fresh_install_check.sh (pip and uv legs)

				which is documented.	
CHG-02	1	done	requirements.txt becomes a full lock compiled from setup.py. Module 4's annotations move into setup.py.	uv pip sync installs exactly the file and resolves nothing, so a top-level-only file builds a broken environment. One source (setup.py) and one generated artifact cannot drift. Snyk sees the full tree.	test_packaging.py::test_lock_pins_every_line, ::test_lock_includes_tooling, ::test_lock_includes_transitive_runtime, ::test_lock_satisfies_setup_py_ranges; scripts/fresh_install_check.sh uv leg; CI uv leg (Phase 9)
CHG-03	2	done	DB_* env contract, DATABASE_URL override, PG* removed, role-aware get_db_config(role=)	The assignment names DB_*. Two parallel contracts invite a misconfigured run that silently uses the wrong account. Role selection makes the owner/app split explicit at every call site.	test_config.py::test_precedence_*, ::test_owner_role_reads_owner_vars, ::test_missing_var_names_variable_not_value, ::test_pg_variables_are_no_longer_read, ::test_no_source_file_reads_a_pg_variable, ::test_env_example_lists_every_variable_the_code_reads
CHG-04	2	done	Connection errors logged sanitized instead of printing the raw exception	libpq errors can echo host and user. Printing them to stdout puts connection details in logs and CI output.	test_config.py::test_connection_error_message_is_sanitized
CHG-05	3	done	query_data.py: f-string SQL replaced by builders returning (sql.Composed, params) and a single executor. Constants become bound params.	The rubric bans f-string SQL outright, and "constants only" is a policy reviewers cannot verify at a glance. Separating build from execute makes every statement inspectable offline and every execution path one function.	test_sql_guard.py::test_no_sql_string_building, ::test_every_execute_receives_composab ::test_guard_reports_sql_built_from_strings; test_query_data.py::test_builder_snapshot ::test_parity_with_module_4, ::test_builders_put_every_value_in_the_parameters_not_the_sql_text
CHG-06	3	done	INSERT and DDL composed from one COLUMNS tuple via Identifier and Placeholder	Two hand-maintained column lists (INSERT text and record mapping) can drift silently. Generating both from one tuple removes the class of bug.	test_load_data.py::test_insert_builder_columns_match_placeholders, ::test_record_mapping_covers_exactly_the_table_columns, ::test_insert_round_trip
CHG-07	3	done	db_safety.py: clamp_limit, parse_limit, validate_text. LIMIT on every SELECT (SQL and ORM).	Unbounded reads are a resource-exhaustion vector, and the rubric requires a ceiling. Centralizing the clamp gives one tested definition of "max". Aggregates get an output LIMIT, not an evaluation LIMIT, so analysis answers are unchanged.	test_db_safety.py::test_clamp_boundaries, ::test_parse_limit_rejects, ::test_validate_text_rejects_nul_without_choing_it, ::test_validate_text_rejects_control_characters_and_surrogates; test_sql_guard.py::test_every_select_has_limit, ::test_every_orm_statement_builder_is_limited, ::test_every_select_the_orm_executes_ha
CHG-08	4	done	New GET /api/applicants: whitelisted sort, enum order, parameterized filters, clamped limit, strict param set	No user input reached SQL, so the defenses had nothing to defend. A read-only JSON endpoint is the smallest surface that exercises every rubric	test_applicant_search.py::*, test_sqli_malicious.py::*

				control and is fully testable.	
CHG-09	4	done	create_app(services: Services, *, database_url, testing). The Services dataclass replaces five keyword arguments and adds search.	Adding a sixth injected dependency would make create_app an unbounded argument list (R0913). One typed seam keeps the Module 4 injection pattern and gives the defaults one home.	test_flask_page.py::test_services_default_to_real_implementations, ::test_services_override_is_used, ::test_services_is_frozen_and_every_field_is_optiona
CHG-10	6	done	Pull route: named exception tuple, busy flag cleared in finally, JSON 500 handler	Narrowing the broad except alone would let an unlisted error leave busy=True forever, turning one failure into a permanent 409. finally makes the state correct regardless.	test_buttons.py::test_unlisted_exception_clears_busy, ::test_unhandled_error_returns_json_500, ::test_every_anticipated_failure_is_a_json_500_and_clears_busy, ::test_a_successful_pull_after_a_failure_is_accepted
CHG-11	5	done	DDL removed from the runtime pull path. Schema setup is owner-only via load_data.py.	The least-privilege app role cannot CREATE or COMMENT, so runtime DDL would break Pull Data. Schema changes are a deployment act, not a request-time act.	test_least_privilege.py::test_load_records_issues_no_ddl, ::test_pull_succeeds_as_app, ::test_pull_route_succeeds_end_to_end_as_app_role, ::test_the_app_role_cannot_create_the_table_if_it_is_missing; test_pull_pipeline.py::test_load_records_does_not_create_the_table, ::test_load_records_maps_a_privilege_error_to_a_clear_message
CHG-12	5	done	Two roles, roles.sql, grants.sql, migrate_ownership.sql	Module 4 ran as a superuser, so any injection would have had full control. The app role gets SELECT and INSERT on one table, which caps blast radius to reading and adding public applicant rows.	test_least_privilege.py::test_role_attributes, ::test_table_privileges, ::test_ddl_deni, ::test_the_owner_owns_the_table_and_the_app_does_not, ::test_public_has_no_privileges_on_the_table, ::test_the_app_role_cannot_create_in_the_schema, ::test_a_grant_by_the_app_role_is_ign test_database_scripts.py::test_roles_sql_takes_verifiers_from_the_environment_not_pas, ::test_verifier_matches_an_independent_computation_from_rfc_5802, ::test_the_documented_setup_never_passes_a_clear_text_password_to_psql; test_gate_checkers.py::test_leak_check_finds_a_password_that_was_committed_and_later_
CHG-13	5	done	Test harness split: TEST_DATABASE_URL (app) and TEST_ADMIN_DATABASE_URL (owner). TEST_DATABASE_URL itself was pulled forward into Phase 2 on purpose (amendment A2.3); Phase 5 adds the admin URL and the app/owner split.	Tests that run as a superuser prove nothing about least privilege. Fixtures need TRUNCATE, which the app must not have.	test_least_privilege.py::test_app_fixture_connects_as_app_role, ::test_owner_fixture_connects_as_owner_role, ::test_the_application_database_url_is_not_the_owner_url
CHG-14	6	done	scrape helpers used by pull_data promoted to public names	They were already a cross-module API. The underscore was a false signal and the cause of W0212.	Existing test_scrape.py and test_pull_pipeline.py tests (renamed), test_scrape.py::test_public_api_surface
CHG-15	6	done	scrape_data(window: ScrapeWindow, options: ScrapeOptions); page loop extracted	Nine positional arguments are an error-prone call surface (R0913/R0914). Dataclasses name every knob and keep defaults in one place.	test_scrape.py::test_options_defaults, ::test_window_and_options_are_immutable, ::test_scrape_data_closes_the_browser_even_when_a_page_raises, existing scrape tests updated
CHG-16	6	done	SQLAlchemy functions imported from sqlalchemy.sql.functions. Quoted sessionmaker[Session] annotations.	Clears 22 astroid false positives without disables. There is no behavior change, and compiled SQL is identical.	test_orm_queries.py::test_compiled_sql_unchanged (Phase 0 snapshot plus exactly the Phase 3 LIMIT; amendment A6.1)

CHG-17	6	done	<code>.pylintrc</code> (minimal), <code>.editorconfig</code> , line-length and newline fixes	Configuration states where the code lives, not what rules to relax. Formatting is enforced by the editor rather than remembered.	<code>test_lint_policy.py::test_pylintrc_relaxes_nothing, ::test_no_inline_disables, ::test_pylint_scores_ten_with_no_messages</code>
CHG-18	0	done	<code>pytest.ini</code> : security marker, <code>--strict-markers</code> , collection hook rejecting unmarked tests. The full suite runs with no marker selection.	Module 4 selected tests by marker expression, so a test carrying only a new marker would silently never run. Running everything and rejecting unmarked tests closes that gap.	<code>test_lint_policy.py::test_every_collected_test_is_marked, ::test_unmarked_items_reports_only_tests_without_an_allowed_marker, ::test_collection_hook_rejects_an_unmarked_test_by_name</code> ; <code>scripts/gate.sh</code> G9 unmarked-dum
CHG-19	9	done	<code>.github/workflows/ci.yml</code> , four jobs; <code>tests.yml</code> untouched	Shift-left: lint, dependency graph, supply chain, and tests each fail independently and visibly. Module 4's workflow keeps that module's grade evidence intact.	Green CI run (<code>actions_success.png</code>); <code>test_ci_config.py::test_four_jobs, ::test_pylint_fail_ur, ::test_svg_validation_step, ::test_the_snyk_job_scans_through_the_marker_aware_script_and_fails_on_findings, ::test_the_test_job_runs_on_both_installers, ::test_ci_never_passes_a_cleartext_password_to_the_role_script</code>
CHG-21	8	done	<code>ROLE_ENV</code> maps each role to a (user variable, password variable) pair instead of a dict with a "password" key holding a string	Snyk Code flagged "password": "DB_PASSWORD" as a hardcoded password. It was a false positive (the string is an environment variable's name), but the shape invited the misreading, for people as well as scanners. The pair expresses what the table means and removes the pattern. There is no behavior change.	<code>test_config.py::test_role_env_maps_each_role_to_a_user_and_password_variable, ::test_no_password_key_holds_a_string_literal_in_src, ::test_env_example_lists_every_variable_the_code_reads</code>
CHG-20	11	planned	RTD repointed to <code>module_5</code> ; Module 4 preserved via tag <code>module-4-final</code>	Docs should describe current code. The tag keeps the Module 4 docs reachable while that grade is pending.	Evidence: RTD build log, both versions load